

模块六: Web 应用架构

第二十一讲: 互联网应用开发基础——理解 C/S 架构与 API

[解释]

本讲概览

本讲是 Web 开发的入门课。你将了解浏览器和服务器之间是如何协作的（客户端-服务器架构），学会用 F12 开发者工具观察网络请求，理解 JSON 数据格式，并用 Python 的 `requests` 库编写脚本调用外部 API。

一、从孤岛到服务

你的代码很强大，但它出不了这台电脑

[解释]

过渡说明

之前的课程中，我们已经学会了用 Python 处理数据和调用 AI 接口。但这些脚本只能在本地终端运行。从本讲开始，我们将学习如何让程序通过网络与外部服务交互，以及如何把自己的程序变成网络服务。

课程回顾：从个人工具到孤岛困境

前五个模块的强大能力

- 模块一：AI 指挥入门，Markdown 文档与 SSOT 思维。
- 模块二/三：Python 基础，变量、函数、结对纠错，建立了 `CLAUDE.md` 标准认知环境。
- 模块四/五：数据采集、清洗、可视化，产出了完整的数据研究报告。



但有一个痛点：孤岛效应

- 只能自己用：你的程序离不开这台电脑。
- 分享难：别人想用，得先装 Python、装库、配环境。
- 交互弱：命令行界面对普通人不友好。

[解释]

孤岛效应

之前写的 Python 脚本只能在自己电脑上运行，发给别人还可能因为环境不同（操作系统、Python 版本、依赖库）而无法执行。Web 应用把计算放在服务器上完成，用户只需要一个浏览器就能使用——这从根本上消除了环境依赖问题，也是 Web 技术至今仍然重要的核心原因。

进阶之路：Web 化解决方案

模块六的解法：Web 化

把 Python 的逻辑 (模块 2-5)
装进
Web 的外壳 (浏览器界面)
= Web 应用

- 随时访问：有浏览器就能用。
- 美观：和现代 App 一样的体验。
- 协作：把你的工具变成云端服务，全世界都能调用。



[解释]

网站 vs Web 应用

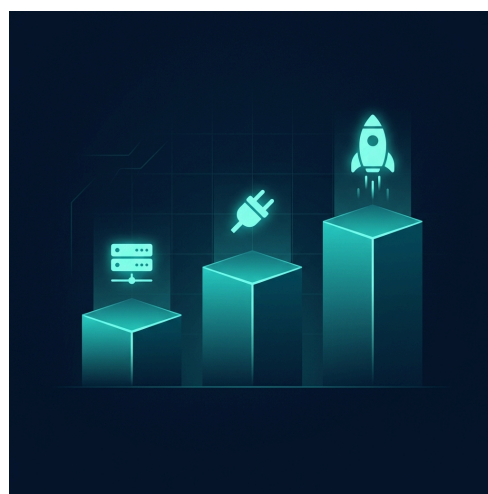
这里需要区分两个概念：

- **网站 (Website)**：像报纸。主要是给你"看"的，比如学校官网、新闻网。内容一般不变。
- **Web 应用 (Web App)**：像工具。主要是给你"用"的，比如 12306 买票、在线修图、Google 文档。它会根据你的操作进行计算。

我们这节课要做的是后者——能干活的网页工具。

本课学习目标

1. **架构认知**：理解现代 Web 应用的 **客户端-服务器 (C/S)** 架构，知道浏览器和服务器是怎么配合的。
2. **剖析数据**：学会用 F12 开发者工具截获请求，读懂互联网传输数据的标准格式——**JSON**。
3. **主动索取**：告别单纯的网页浏览，用 Python `requests` 编写脚本，主动调用外部接口获取结构化数据。



核心类比：餐厅模型

本讲用餐厅场景来类比 Web 架构中的各个角色，帮助理解抽象概念。

[解释]

请求与响应 (Request & Response)

这是互联网世界最基本的对话方式，就像一问一答。

- **Request (问)**：浏览器说"我要看菜单"。
- **Response (答)**：服务器说"这是菜单数据"。
不管多复杂的网站，哪怕是双十一秒杀、在线视频会议，本质上也是由无数个这样的"一问一答"组成的。

二、互联网是怎么工作的

用餐厅类比理解互联网架构

[解释]

类比说明

本讲用餐厅场景来类比 Web 架构中的各个角色：顾客对应浏览器（客户端），后厨对应服务器（后端），菜单对应 API（接口），服务员对应 HTTP 协议（通信规则）。理解这个类比后，后续的技术概念会更容易接受。

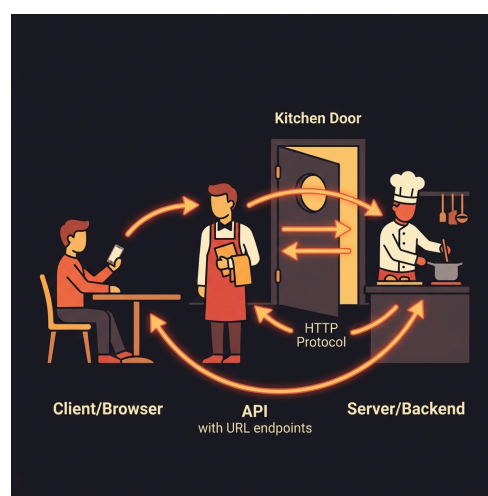
角色映射：餐厅类比

顾客和后厨互不相见，怎么配合？

顾客（浏览器）和后厨（服务器）之间需要一套标准化的协作机制：

角色映射

1. 顾客 (Client/浏览器)：负责提要求 (点餐)。
 - 你的浏览器、手机 App
2. 后厨 (Server/服务器)：负责干活 (做菜)。
 - 运行你 Python 代码的电脑
3. 菜单 (API)：唯一的沟通标准。
 - URL 接口列表
4. HTTP 协议：负责在客户端与服务器之间传递请求和响应。



[解释]

前后端分离架构

现代 Web 应用通常采用“前后端分离”的架构。为什么要分开？

想象装修房子：硬装（后端）是水电管线、承重墙，要稳固不轻易改动；软装（前端）是壁纸、窗帘、家具，可以经常换风格。

把它们分开，你就可以在不砸墙（不改后端代码）的情况下，随意换窗帘（改网页界面）。各司其职，互不干扰。

通信规则：HTTP 协议

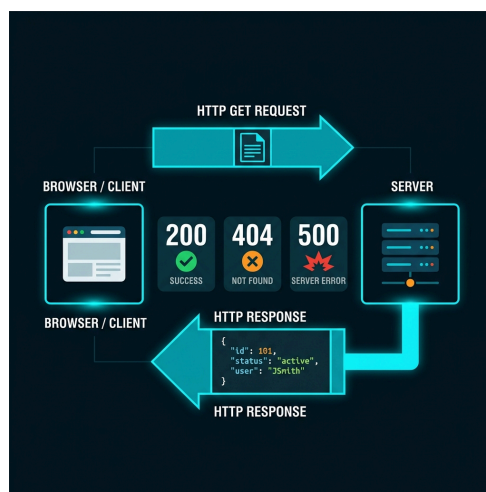
两个最常用的"动作指令"

浏览器和服务器的对话，必须遵守规则。

1. **GET (看菜单)**：我想要获取信息。
 - `GET /menu` → 给我看看有什么菜
2. **POST (提交)**：向服务器提交数据进行处理。
 - `POST /order` → 我要点一份红烧肉

状态码（服务器的反馈）

- **200 OK**：菜做好了 ✅ (成功)。
- **404 Not Found**：没这道菜 ❌ (网址错了)。
- **500 Server Error**：服务器内部出错 💣 (代码报错)。



生活常识升级：以后大家打不开网页时，看看浏览器显示的是 404 还是 500——前者是"地址写错了"，后者是"服务器出故障了"。

[解释]

什么是协议 (Protocol)

协议就是"沟通的规矩"。就像打电话，我说"喂"，你知道要回答"你好"；我说"再见"，你知道要挂电话。HTTP 协议就是浏览器和服务器的通话规矩。GET 的意思是"给我看看"，POST 的意思是"请帮我处理这个"。

状态码是服务器端的标准回复代码，业界统一使用。200 表示成功，4xx 系列表示"你的请求有问题"（客户端错误），5xx 系列表示"我这边出了问题"（服务器错误）。

亲眼看见"一问一答"：F12 Network 实操

动手实验

1. 打开 Chrome 浏览器，访问任意网站（如学校官网）。
2. 按 **F12** 打开开发者工具。
3. 切换到 **Network** 选项卡。
4. 刷新页面，观察瀑布式的请求列表。

你会看到什么？

- 每一行就是一次 **Request** → **Response**。
- 点击任意一行，可以看到：
 - **Method**：GET 还是 POST？
 - **Status**：200？404？304？
 - **Response**：服务器实际返回了什么数据？

观察要点

```
General:
  Request URL: https://www.ncu.edu.cn/
  Request Method: GET
  Status Code: 200 OK

Response Headers:
  Content-Type: text/html; charset=utf-8

Response Body:
<!DOCTYPE html>
<html>...
```

💡 **互联网的工作方式**：你看到的每一个网页，都是浏览器发送了一个 GET 请求，服务器返回了一段 HTML 文本，浏览器再把它"画"成了页面。

[解释]

为什么要动手抓包

通过 F12 开发者工具亲自观察网络请求，可以直观理解浏览器在后台做了什么：每次页面加载、每次点击按钮，浏览器都在向服务器发送请求并接收响应。这是理解 Web 应用工作原理的基础。

互联网的通用语：JSON

它是谁？

- 全称：JavaScript Object Notation。
- 地位：互联网世界的通用语。
- 作用：让 Python (后端) 和浏览器 (前端) 能听懂彼此说话。

秒懂指南

你不需要专门学 JSON，因为：

1. 还记得 Python 字典 吗？
`{"key": "value"}`
2. JSON 长得和 Python 字典几乎一模一样！

结论：只要你在 Python 里返回一个字典，FastAPI 自动把它翻译成 JSON 发给浏览器。

对比与真实体验

```
# Python 字典
menu = {"麻婆豆腐": 12, "宫保鸡丁": 15}
```

```
// JSON (服务器发出的格式)
{"麻婆豆腐": 12, "宫保鸡丁": 15}
```

亲眼看看真实的 JSON：

南昌现在的气温是多少度？无需写代码，直接点击下方这个无需注册的公共 API 接口，看看它返回的 JSON 数据：

[点击获取南昌实时天气 JSON](#)

[解释]

JSON：机器世界的世界语

互联网上有各种各样的编程语言 (Python、Java、JavaScript)，它们之间怎么交流？就需要一种大家都能看懂的格式，这就是 JSON。

它非常简单，就是一堆“键值对”，长得几乎和 Python 字典一模一样。所以不需要额外学新语法——会写字典就会写 JSON。

FastAPI 在这里做了一层自动转换：你在 Python 里 return 一个 dict，它帮你序列化成 JSON 字符串并设置好 Content-Type 头。

三、主动获取数据： 用代码调用 API

用 Python 代码调用外部 API 获取数据

[解释]

为什么用 Python 而不继续用浏览器

浏览器虽然能请求 API，但无法对返回的数据做进一步处理。通过 Python 的 `requests` 库发起请求，我们可以将返回的 JSON 数据存入变量，进行过滤、计算、写入文件等后续操作。这是从“手动查看”到“程序化处理”的关键一步。

实操一：获取城市坐标 (Geocoding)

业务需求

- Open-Meteo 的核心气象预报接口只接受经度和纬度，不接受如 "Nanchang" 这样的城市名称文本。
- 因此我们需要分两步走：先调用它专门的地理编码接口 (Geocoding API)，把城市名“翻译”成坐标。

库的准备： requests

requests 是 Python 中最常用的 HTTP 请求库。

```
# 在终端安装 requests 库  
pip install requests
```

参考提示词（发给 AI 助手）

"请帮我用 Python 的 requests 库，向 `https://geocoding-api.open-meteo.com/v1/search?name=Nanchang&count=1` 发起 GET 请求，把返回的 JSON 文本解析为字典并打印。"

AI 生成的参考代码

```
import requests  
  
# 1. 向目标 API 发起 GET 请求  
url = "https://geocoding-api.open-meteo.com/v1/search?name=Nanchang&count=1"  
response = requests.get(url)  
  
# 2. 将返回的 JSON 文本解析为 Python 字典  
geo_data = response.json()  
print(geo_data)
```

[解释]

response.json()

response.json() 会将服务器返回的 JSON 文本自动转换为 Python 字典。这是将互联网数据接入 Python 程序的关键桥梁，从这一步开始，数据变成了代码可直接操作的变量。

实操二：JSON 数据深度解析

观察：返回的嵌套结构

终端打印出的 `geo_data` 结构如下：

```
{
  "results": [
    {
      "id": 1806330,
      "name": "Nanchang",
      "latitude": 28.68333,
      "longitude": 115.88333,
      "country": "China"
    }
  ]
}
```

如何提取目标字段？

在字典的层叠嵌套中，要拿到经纬度，我们需要像剥洋葱一样层层深入：

1. 第一层提取：

```
geo_data['results']
```

- 拿到一个中括号 `[]` 数组。

2. 第二层提取：

```
geo_data['results'][0]
```

- 拿到数组里第一项的大括号 `{}` 字典。

3. 第三层提取：取出经纬度字段。

```
# 精准层层提取
lat = geo_data['results'][0]['latitude']
lon = geo_data['results'][0]['longitude']
print(f"提取成功，南昌坐标:纬度{lat}，经度{lon}")
```

剥洋葱法则：遇到 `[]` 就用数字（如 `[0]`）取索引位置，遇到 `{}` 就用键名（如 `['name']`）取对应值。

[解释]

结构化解析

在真实的开发中，API 返回的数据结构往往非常复杂。结合 F12 工具能帮助开发者清晰地确认访问路径的键名。掌握“剥洋葱”原理，是将粗放数据转为业务可用变量的必经之路。

实操三：高阶组合——两步串行请求

工作流的完整大串联

- 真实工作流中，API 之间往往是**接力**的。
- **步骤 A**：将“地名词”转化为“经纬度”。
- **步骤 B**：拿着“经纬度”去请求气象站，获取当下的天气。



HOTL 实践指导

你可以把多步逻辑写得很清楚，让 AI 直接生成：“请先请求接口 A 提取坐标...然后把坐标拼接到接口 B 的 URL 中访问提取温度”。

参考提示词（发给 AI 助手）

"请用 Python 的 requests 库帮我写一段两步走的串行请求代码：

第一步：请求地理编码接口

`https://geocoding-api.open-meteo.com/v1/search?`

`name=Nanchang&count=1`，从中解析出 latitude 和 longitude。

第二步：将这两个变量拼接到天气接口 URL 中：

`https://api.open-`

`meteo.com/v1/forecast?`

`latitude={lat}&longitude=`

`{lon}¤t_weather=true`，

发起请求并从中提取

`current_weather` 里的

`temperature`，然后打印出来。"

AI 生成的参考代码

```
import requests

# 【步骤 A：查坐标】
geo_url = "https://geocoding-api.open-meteo.com/v1/search?name=Nanchang&count=1"
geo_res = requests.get(geo_url).json()
lat = geo_res['results'][0]['latitude']
lon = geo_res['results'][0]['longitude']

# 【步骤 B：查天气（将坐标接入新网址）】
weather_url = f"https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}&current_weather=true"
weather_res = requests.get(weather_url).json()

# 【最终提取气温】
temp = weather_res['current_weather']['temperature']
print(f"数据走通！当前实时温度为：{temp}°C")
```

[解释]

API 编排与通信接力

这种“调用接口A拿结果拼凑参数去调用接口B”的模式在真实开发中极其频繁。理解这个接力过程，也是理解如何通过外部接口驱动系统运作的基础架构思维。

本讲小结与下讲预告

你学会了什么（作为消费者）

1. **架构**：理解了浏览器（客户端）、服务器（后端）与 API（接口）的协作关系。
2. **语言**：能识别互联网数据传输的标准格式——看懂 **JSON** 的键值对结构。
3. **透视**：学会利用 **F12 Network** 面板抓取请求轨迹，并用 `requests` 库直接在代码中调用了外部 API。

下节课预告：角色切换（作为创造者）

“目前我们只会调用别人提供的 API，角色是数据的消费者。当你有能力自己对外提供 API 时，你就成为了数据的生产者。”

- **下节看点**：我们将学习 Python Web 框架 **FastAPI**，搭建自己的后端服务。
- 把你刚才获取到的气象数据，封装成可以通过浏览器访问的 Web 服务。

[解释]

调用方与提供方

本讲中我们始终处于“调用方”角色——向别人的服务器发请求、拿数据。下一讲将切换到“提供方”角色：用 FastAPI 编写自己的后端接口，让别人的浏览器来访问我们的数据。